

Tendências de Motores de Jogos

# Semana 8

Animation Blueprint, State Machine, Blend Spaces e Montages

**Disciplina:** Tendências de Motores de Jogos (IN46A)

**Instituição:** IFMS — Campus Dourados

**Curso:** Tecnologia em Jogos Digitais

**Módulo 3 — Resolver Problemas**

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

## Objetivos da Semana

### OBJETIVOS

- Compreender Animation Blueprint e State Machine como mecanismo universal de transição de estados de animação
- Implementar uma State Machine básica (idle, andar, correr) sobre o `BP_Player` já existente
- Compreender Blend Spaces como interpolação multidimensional e Montages como animações pontuais sobrepostas
- Propor e implementar, com autonomia própria, uma animação contextual escolhendo a ferramenta adequada ao problema

## Como a semana está organizada

---

- **Encontro 1**  
Animation Blueprint e State Machine — transição entre idle, andar e correr
- **Encontro 2**  
Blend Spaces e Montages — desafio: animação contextual à escolha do grupo

Tendências de Motores de Jogos

ENCONTRO 1

# Animation Blueprint e State Machine

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



# O seu personagem anda desde o Módulo 1. Mas ele *parece* andar?

Pense na diferença entre o `BP_Player` saber que está em movimento e o jogador ver isso na tela.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

## Separar decisão de gameplay e apresentação visual

- O `BP_Player` já sabe se está parado, andando ou correndo — isso é velocidade processada pelo Character Movement Component
- Falta uma camada que traduza esse estado interno em uma animação visível
- Misturar essa decisão com a lógica de movimento acoplaria apresentação a gameplay
- Toda engine madura resolve isso com uma camada separada, dedicada apenas a animação

### DICA

Trocar o esqueleto ou o conjunto de animações nunca deve exigir tocar na lógica de input do personagem.

## Animation Blueprint: a camada de decisão

- Asset separado do `BP_Player`, associado ao esqueleto do personagem
- Lê variáveis expostas pelo Character (como velocidade)
- Decide qual animação reproduzir a cada instante
- Nunca contém lógica de gameplay — apenas lógica de apresentação

## State Machine: estados e transições

- Estrutura de estados discretos (Idle, Walk, Run) conectados por regras de transição
- Cada transição é lida a partir de uma variável de gameplay (velocidade), nunca arbitrária
- Aplicada hoje aos estados de locomoção já existentes no `BP_Player`
- Cada seta da State Machine é uma condição verificável, não uma decisão livre



## Transição de animação: Unreal × Unity

### Unreal

Animation Blueprint com State Machine, acessando propriedades do Character diretamente via nós de Blueprint

### Unity

Animator Controller, acionado por parâmetros expostos no script ( `SetFloat` , `SetBool` , `SetTrigger` )

## Do dado de gameplay à animação



IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

## **Demonstração: State Machine de três estados**

O professor cria um Animation Blueprint associado ao esqueleto do personagem, expõe a velocidade do `BP_Player` como variável lida pela State Machine, e constrói os estados Idle, Walk e Run com transições por faixas de velocidade.

**Resultado esperado:** transições visuais corretas entre os três estados, sem interromper o fluxo jogável do Módulo 2.

### *Imagem sugerida*

*Objetivo: mostrar a estrutura visual de uma State Machine dentro do Animation Blueprint, reforçando a ideia de estados discretos conectados por transições condicionais.*

*Enquadramento: captura de tela do editor de State Machine da Unreal Engine, com três blocos de estado visíveis.*

*Elementos presentes: blocos "Idle", "Walk" e "Run" conectados por setas de transição; uma das setas com a condição de velocidade exposta em destaque.*

*Destaque visual: a seta de transição em avaliação, com a condição de velocidade visível em primeiro plano.*

*Legenda sugerida: "Cada seta é uma condição verificável, nunca uma decisão livre."*

## Boas práticas

### ✓ BOAS PRÁTICAS

- Nomear estados da State Machine de forma explícita (Idle, Walk, Run — nunca State\_01, State\_02)
- Verificar que as faixas de velocidade das transições não se sobrepõem nem deixam lacunas
- Manter o Animation Blueprint livre de qualquer lógica de gameplay

# Laboratório do Encontro 1

Implementar uma State Machine básica (Idle, Walk, Run) no Animation Blueprint do próprio BP\_Player , validando visualmente a transição entre os três estados.

## OBJETIVOS

Critério de sucesso: transições fluidas e corretas, sem interromper o fluxo jogável consolidado na Semana 7.

## Fechando o Encontro 1

---

- Animation Blueprint separa decisão de animação da lógica de gameplay; State Machine organiza estados e transições
- BP\_Player de cada grupo exibindo transições corretas entre idle, andar e correr
- Próximo encontro: dois problemas novos de animação — interpolação contínua e ação pontual sobreposta

Tendências de Motores de Jogos

## ENCONTRO 2

# Blend Spaces e Montages

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos



Tendências de Motores de Jogos



**Andar para os lados é um novo estado, ou uma variação do estado que já existe?**

Pense se movimentos direcionais deveriam virar novos blocos na State Machine.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

## Dois problemas parecidos, duas soluções distintas

- Combinar continuamente animações conforme uma direção não é o mesmo que sobrepor uma ação pontual a um estado em curso
- Uma State Machine de estados discretos não resolve bem interpolação contínua
- Uma ação pontual (dano, ataque, interação) não deve alterar permanentemente o estado da State Machine
- A engine precisa de uma ferramenta para cada um desses dois problemas

### DICA

Contínuo e direcional pede Blend Space; pontual e sobreposto pede Montage — a distinção é a base do desafio de hoje.

## **Blend Space: interpolação multidimensional**

- Interpola entre animações-base de acordo com coordenadas (direção, velocidade)
- Resolve movimentos direcionais (frente, lateral, diagonal) como um espaço contínuo
- Conectado à State Machine já existente, não a substitui
- Configurado visualmente como um espaço 1D ou 2D

## Montage: animação pontual sobreposta

- Reproduzida uma única vez, sobreposta ao estado corrente
- Não altera permanentemente a State Machine — devolve o controle ao final
- Usada para reação a dano, ataque simples ou interação pontual
- Pode reutilizar `BPI_Interactable`, já construído na Semana 5

## Blend Space e Montage: Unreal × Unity

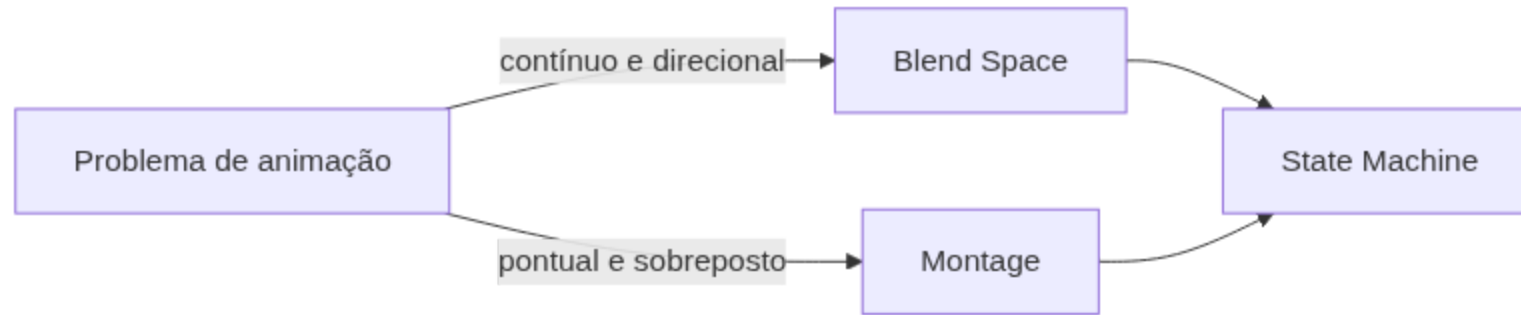
### Unreal

Blend Space multidimensional nativo;  
Montage dedicado para animação pontual  
sobreposta

### Unity

Blend Tree dentro do Animator Controller  
para interpolação; sem equivalente  
nomeado ao Montage — efeito obtido  
combinando Layers e Avatar Masks

## Dois problemas, duas ferramentas



## **Demonstração: Blend Space e Montage**

---

O professor demonstra um Blend Space direcional simples (frente, lateral, trás) associado à State Machine existente, e depois uma Montage disparada por um evento de teste, mostrando que ela sobrepõe a animação corrente sem interromper a State Machine.

**Resultado esperado:** a turma vê os dois problemas resolvidos lado a lado, antes de decidir sozinha qual se aplica ao próprio desafio.

### *Imagem sugerida*

*Objetivo: mostrar a diferença visual entre um Blend Space (espaço contínuo de interpolação) e uma Montage (linha do tempo de animação pontual), reforçando que são ferramentas para dois problemas distintos.*

*Enquadramento: duas capturas de tela lado a lado do editor da Unreal Engine.*

*Elementos presentes: à esquerda, um Blend Space 2D com eixos de direção e velocidade e um ponto de amostragem no meio do espaço; à direita, a linha do tempo de uma Montage com uma única animação sobreposta.*

*Destaque visual: o contraste entre o espaço contínuo à esquerda e a linha do tempo pontual à direita.*

*Legenda sugerida: "Contínuo e direcional à esquerda; pontual e sobreposto à direita."*



## Desafio: animação contextual

Cada grupo propõe e implementa uma animação contextual própria — reação a dano, interação com um objeto do mundo ou um ataque simples — escolhendo entre Blend Space ou Montage conforme a natureza do problema.

### ATENÇÃO

Não há indicação prévia de qual ferramenta usar. A escolha correta faz parte do desafio.

## Boas práticas

### ✓ BOAS PRÁTICAS

- Testar a Montage isoladamente antes de conectá-la a um evento de gameplay
- Nomear eixos do Blend Space de forma explícita (Direção, Velocidade)
- Justificar a escolha da ferramenta a partir da natureza do problema, não da preferência pessoal

## Resumo da Semana 8

---

- Animation Blueprint separa decisão de animação de lógica de gameplay; State Machine organiza estados discretos
- Blend Space resolve interpolação contínua; Montage resolve ação pontual sobreposta
- Primeiro desafio de liberdade real de solução da disciplina, com escolha justificada entre as duas ferramentas
- Unidade III — Resolver Problemas — iniciada

## Checklist da Semana

### OBJETIVOS

- State Machine funcional (Idle, Walk, Run) sobre o BP\_Player
- Blend Space direcional funcional
- Animação contextual própria (Blend Space ou Montage) implementada e justificada
- Nenhum sistema do Módulo 2 substituído ou quebrado

## Próximos passos

### DICA

A Semana 9 introduz UMG e HUD para comunicar em tempo real o estado de jogo ao jogador, reutilizando dados já expostos pela State Machine e o progresso persistido pelo `SaveComponent`. A metodologia permanece Challenge Based Learning: cada grupo escolhe quais dados compõem o HUD e propõe a própria solução de binding.

**Leitura recomendada:** Animation Blueprints in Unreal Engine (Epic Games Documentation).